Abhishek Yadav

UsnNo:CS23071

Lab 5 Extra Datasets

Aim:To implement the Random Forest algorithm on the Gaming and Mental Health dataset and evaluate the model performance using appropriate metrics.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

```
from google.colab import drive
drive.mount('/content/drive')
path = "/content/drive/MyDrive/ML datasets/Gaming and Mental Health.csv"
df = pd.read_csv(path)
df.head()
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

	record_id	age	gender	daily_gaming_hours	game_genre	primary_game	gaming_platform	sleep_hours	sleep_quality	sleep_disruption_fre
0	GD0001	17	Male	11.1	Mobile Games	Clash of Clans	PC	3.7	Very Poor	Son
1	GD0002	21	Male	3.0	MOBA	Dota 2	PC	7.2	Fair	
2	GD0003	23	Male	7.6	FPS	CS:GO	Multi-platform	4.4	Fair	
3	GD0004	20	Female	7.2	RPG	Skyrim	Multi-platform	5.1	Fair	
4	GD0005	18	Male	6.8	Battle Royale	Apex Legends	PC	3.4	Poor	

5 rows × 27 columns

```
df.shape
```

```
(1000, 27)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 27 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   record_id                   1000 non-null   object
 1   age                         1000 non-null   int64
 2   gender                      1000 non-null   object
 3   daily_gaming_hours          1000 non-null   float64
 4   game_genre                  1000 non-null   object
 5   primary_game                1000 non-null   object
 6   gaming_platform             1000 non-null   object
 7   sleep_hours                 1000 non-null   float64
 8   sleep_quality               1000 non-null   object
 9   sleep_disruption_frequency  1000 non-null   object
 10  academic_work_performance   1000 non-null   object
 11  grades_gpa                  754 non-null    float64
 12  work_productivity_score     674 non-null    float64
 13  mood_state                  1000 non-null   object
 14  mood_swing_frequency        1000 non-null   object
 15  withdrawal_symptoms         1000 non-null   bool
 16  loss_of_other_interests     1000 non-null   bool
 17  continued_despite_problems  1000 non-null   bool
 18  eye_strain                  1000 non-null   bool
 19  back_neck_pain              1000 non-null   bool
 20  weight_change_kg            1000 non-null   float64
 21  exercise_hours_weekly       1000 non-null   float64
 22  social_isolation_score      1000 non-null   int64
```

```
 23  face_to_face_social_hours_weekly  1000 non-null   float64
 24  monthly_game_spending_usd         1000 non-null   float64
 25  years_gaming                      1000 non-null   int64
 26  gaming_addiction_risk_level       1000 non-null   object
dtypes: bool(5), float64(8), int64(3), object(11)
memory usage: 176.9+ KB
```

```
df.isnull().sum()
```

	0
record_id	0
age	0
gender	0
daily_gaming_hours	0
game_genre	0
primary_game	0
gaming_platform	0
sleep_hours	0
sleep_quality	0
sleep_disruption_frequency	0
academic_work_performance	0
grades_gpa	246
work_productivity_score	326
mood_state	0
mood_swing_frequency	0
withdrawal_symptoms	0
loss_of_other_interests	0
continued_despite_problems	0
eye_strain	0
back_neck_pain	0
weight_change_kg	0
exercise_hours_weekly	0
social_isolation_score	0
face_to_face_social_hours_weekly	0
monthly_game_spending_usd	0
years_gaming	0
gaming_addiction_risk_level	0

dtype: int64

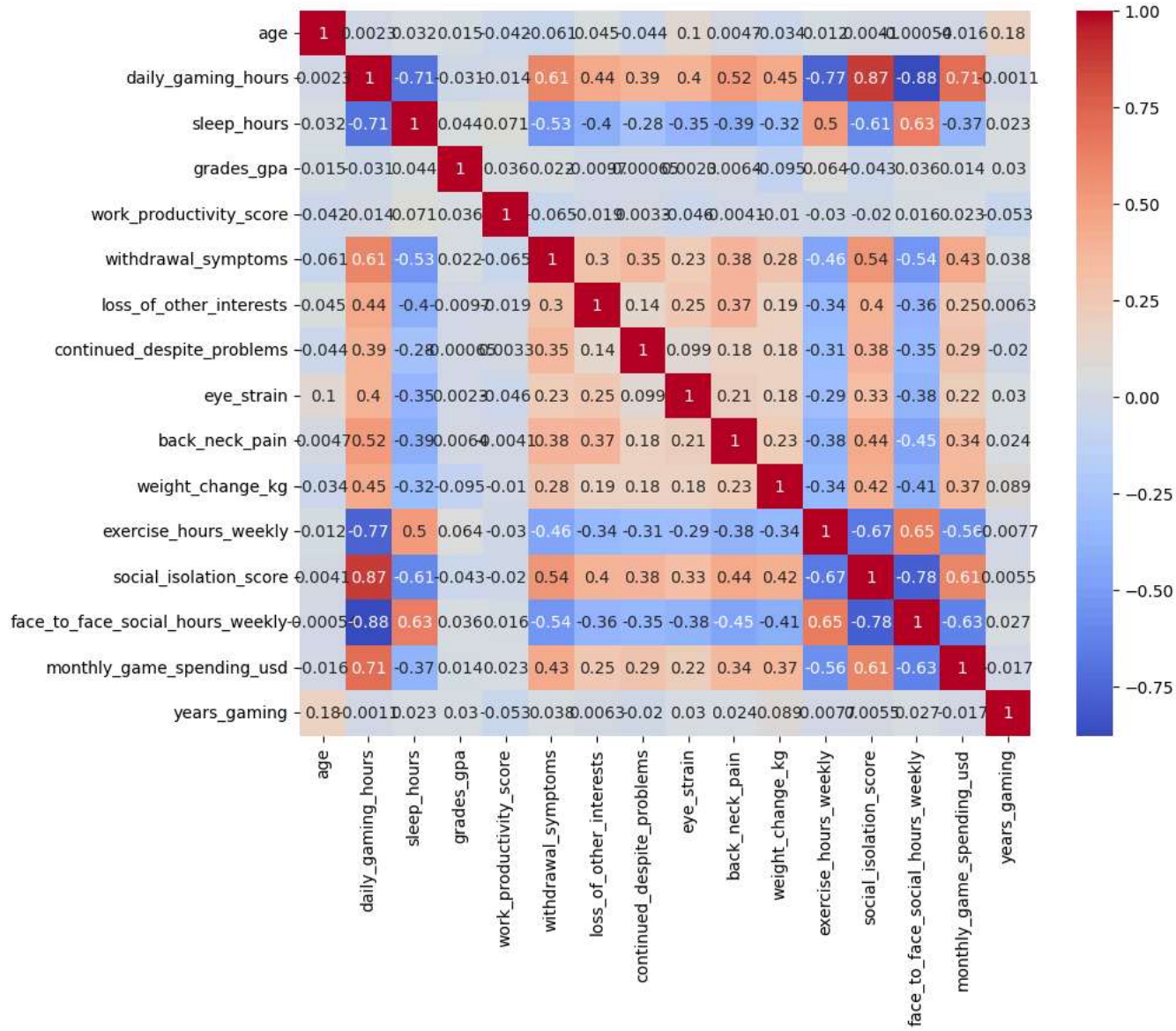
```
df = df.dropna()
```

```
df.fillna(df.mean(numeric_only=True), inplace=True)
```

```
/tmp/ipython-input-795809146.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df.fillna(df.mean(numeric_only=True), inplace=True)
```

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```




```
X = df.drop(columns=['gaming_addiction_risk_level'])   # change target name
y = df['gaming_addiction_risk_level']
```

```
X = pd.get_dummies(X, drop_first=True)
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.linear_model import LogisticRegression
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)
y_pred_log = log_model.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_log))
print(classification_report(y_test, y_pred_log))
```

```
Logistic Regression Accuracy: 0.6395348837209303
              precision    recall  f1-score   support

        High       0.54      0.33      0.41        21
```

```
         Low       0.75      1.00      0.86        40
    Moderate       0.33      0.12      0.17        17
      Severe       0.43      0.75      0.55         8

    accuracy                           0.64        86
   macro avg       0.51      0.55      0.50        86
weighted avg       0.59      0.64      0.59        86
```

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
```

```
Decision Tree Accuracy: 0.9069767441860465
```

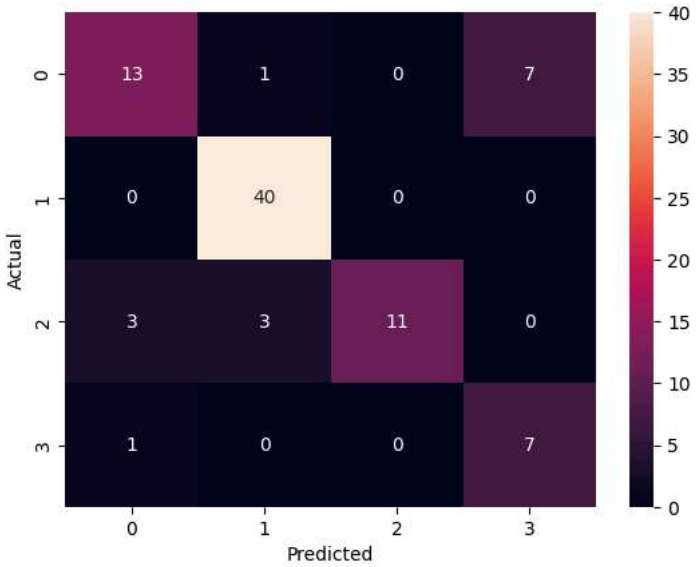
```
rf_model = RandomForestClassifier()
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Random Forest Accuracy: 0.8255813953488372
```

```
from sklearn.neural_network import MLPClassifier
ann_model = MLPClassifier(hidden_layer_sizes=(50,50), max_iter=500)
ann_model.fit(X_train, y_train)
y_pred_ann = ann_model.predict(X_test)
print("ANN Accuracy:", accuracy_score(y_test, y_pred_ann))
```

```
ANN Accuracy: 0.5465116279069767
```

```
cm = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
df = pd.read_csv(path)
df.head()
```

	record_id	age	gender	daily_gaming_hours	game_genre	primary_game	gaming_platform	sleep_hours	sleep_quality	sleep_disruption_fre
0	GD0001	17	Male	11.1	Mobile Games	Clash of Clans	PC	3.7	Very Poor	Son
1	GD0002	21	Male	3.0	MOBA	Dota 2	PC	7.2	Fair	
2	GD0003	23	Male	7.6	FPS	CS:GO	Multi-platform	4.4	Fair	
3	GD0004	20	Female	7.2	RPG	Skyrim	Multi-platform	5.1	Fair	
4	GD0005	18	Male	6.8	Battle Royale	Apex Legends	PC	3.4	Poor	

5 rows × 27 columns

```
df.columns
```

```
Index(['record_id', 'age', 'gender', 'daily_gaming_hours', 'game_genre',
       'primary_game', 'gaming_platform', 'sleep_hours', 'sleep_quality',
       'sleep_disruption_frequency', 'academic_work_performance', 'grades_gpa',
       'work_productivity_score', 'mood_state', 'mood_swing_frequency',
       'withdrawal_symptoms', 'loss_of_other_interests',
       'continued_despite_problems', 'eye_strain', 'back_neck_pain',
       'weight_change_kg', 'exercise_hours_weekly', 'social_isolation_score',
       'face_to_face_social_hours_weekly', 'monthly_game_spending_usd',
       'years_gaming', 'gaming_addiction_risk_level'],
      dtype='object')
```

```
X = df.drop(columns=['gaming_addiction_risk_level'])
y = df['gaming_addiction_risk_level']
```

```
X = pd.get_dummies(X, drop_first=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = RandomForestClassifier()
model.fit(X_train, y_train)
```

```
▾ RandomForestClassifier
RandomForestClassifier()
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.875

Classification Report:

              precision    recall  f1-score   support

        High       0.73      0.63      0.68        38
         Low       1.00      0.98      0.99        99
    Moderate       0.76      0.88      0.82        33
      Severe       0.78      0.83      0.81        30

    accuracy                           0.88       200
   macro avg       0.82      0.83      0.82       200
weighted avg       0.88      0.88      0.87       200
```

```
print("Logistic:", accuracy_score(y_test, y_pred_log))
print("Decision Tree:", accuracy_score(y_test, y_pred_dt))
print("Random Forest:", accuracy_score(y_test, y_pred_rf))
print("ANN:", accuracy_score(y_test, y_pred_ann))
```

```
Logistic: 0.6395348837209303
Decision Tree: 0.9069767441860465
Random Forest: 0.8255813953488372
ANN: 0.5465116279069767
```